

P1747R0

Don't use ``char8_t`` and ``std::u8string`` yet in P1389

Published Proposal, 2019-06-12

This version:

https://yehezekelshb.github.io/cpp_proposals/sg20/P1747-dont-use-char8_t-yet-in-P1389.html

Author:

[Yehezkel Bernat](#)

Audience:

SG20

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

P1747 claims that we shouldn't use ``char8_t`` and ``std::u8string`` in P1389 as of C++20

Table of Contents

1 Problem statement

2 Proposed Wording

3 Acknowledgements

§ 1. Problem statement

P1389, under 2.2.1.1. Primary types ([types.basic.primary]), suggest that post- C++20 we should teach beginners to use `char8_t` for characters and `std::u8string` for strings (instead of `char` and

`std::string` in pre-C++20).

The author thinks this is wrong.

C++20 still has no tools to handle input and output with these types. Even the new `{fmt}` facilities doesn't support it. There is even no good conversion tools for it (and even the existing conversions, like `codecvt` stuff, are deprecated since C++17).

The main usage of strings and characters is for input and output and C++20 still missing tools to do so with these types.

This paper suggests to remove the distinction between pre-C++20 and post-C++20 and reintroduce these types as soon as the proper tools are added (by SG16, hopefully for C++23).

§ 2. Proposed Wording

Under 2.2.1.1. Primary types ([types.basic.primary]):

Abstract type	Pre- C++ 20 type	Post-C++20 type
Integer	<code>int</code>	<code>int</code>
Floating-point	<code>double</code>	<code>double</code>
Boolean	<code>bool</code>	<code>bool</code>
Character	<code>char</code>	<code>char8_t</code>
String	<code>std::string</code>	<code>std::u8string</code>
Sequence container	<code>std::vector</code>	<code>std::vector</code>
Associative container	<code>std::map</code>	<code>std::map</code>

~~The distinction between pre-C++20 and C++20 is simply the acknowledgement of UTF-8. This is not to suggest that students should be introduced to the details of UTF-8 any earlier, but rather to get the idea of UTF-8 support on their radar, so that when they need to care about locales, they won't need to shift from thinking about why `char` is insufficient in the current programming world: they can just start using what they are already familiar with.~~ It may worth to warn the students that the support for non-English locales may vary depends on the specific platform.

§ 3. Acknowledgements

Thanks for Christopher Di Bella for mentioning the point about `{fmt}`